sas innovate

Options for Analyzing Database Tables Using SQL and FedSQL

Allison Saito
Sr Technical Training Consultant

Special thanks to Peter Styliadis for writing the original version of this workshop

#SASinnovate

Contents

Abstract **2**

Data Dictionary **3**

Slides **4**

Hands On Workshop Instructions **5**

Setup: Sign In and Create Data *5*

PROC SQL and SAS Compute Server Database Processing *6*

PROC FedSQL and SAS Compute Server Database Processing *16*

CAS Server Database Processing *20*

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

Abstract

Learn different ways to access and analyze database tables using SQL on the SAS® Viya® platform. First, we focus on using the familiar SQL procedure in SAS® Viya® to work with database tables. Then we focus on accelerating your SQL queries in SAS® Viya® using the SAS® Cloud Analytics Services (CAS) engine, which is the high-performance, massively parallel processing environment on the SAS® Viya® platform.

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

Data Dictionary

We will work with a data set named **LOANS_RAW**. The data consists of information about various loans taken out by fake customers of a fictional bank. We will use it to generate summary statistics.

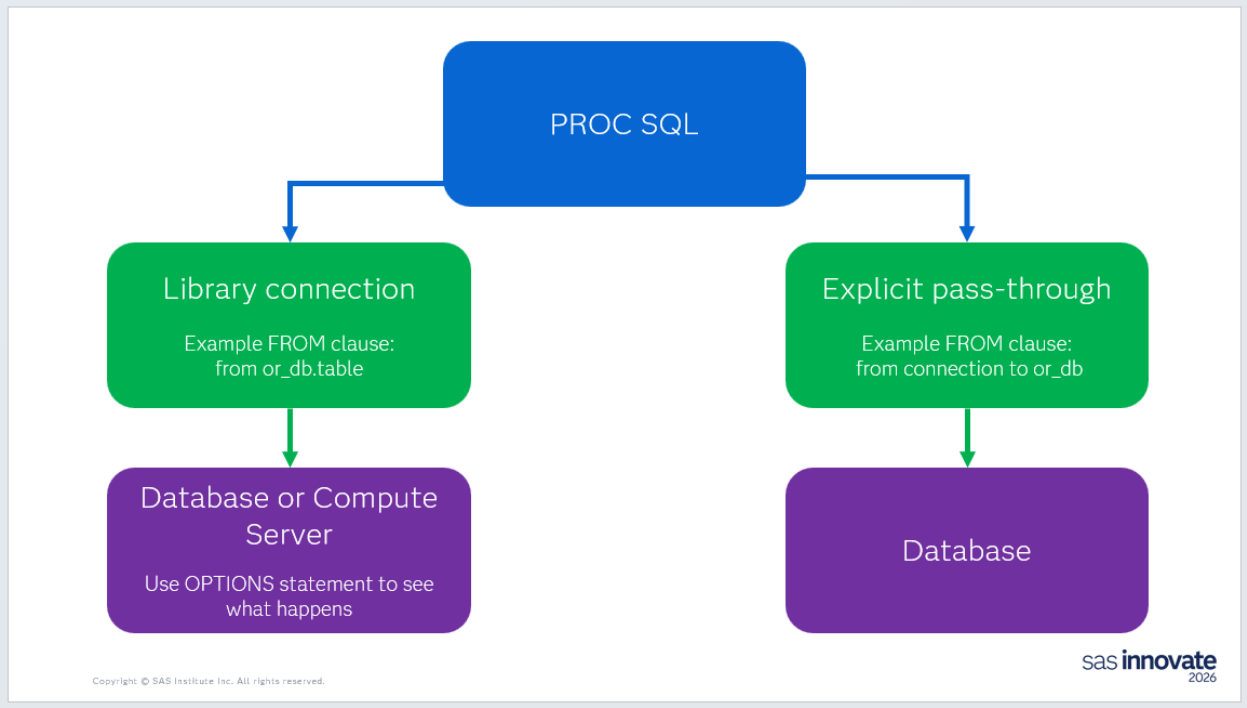
Each row of the data set contains a different loan. Each loan is uniquely identified by **AccNumber**. While many columns appear in the data, the following are most relevant to this workshop.

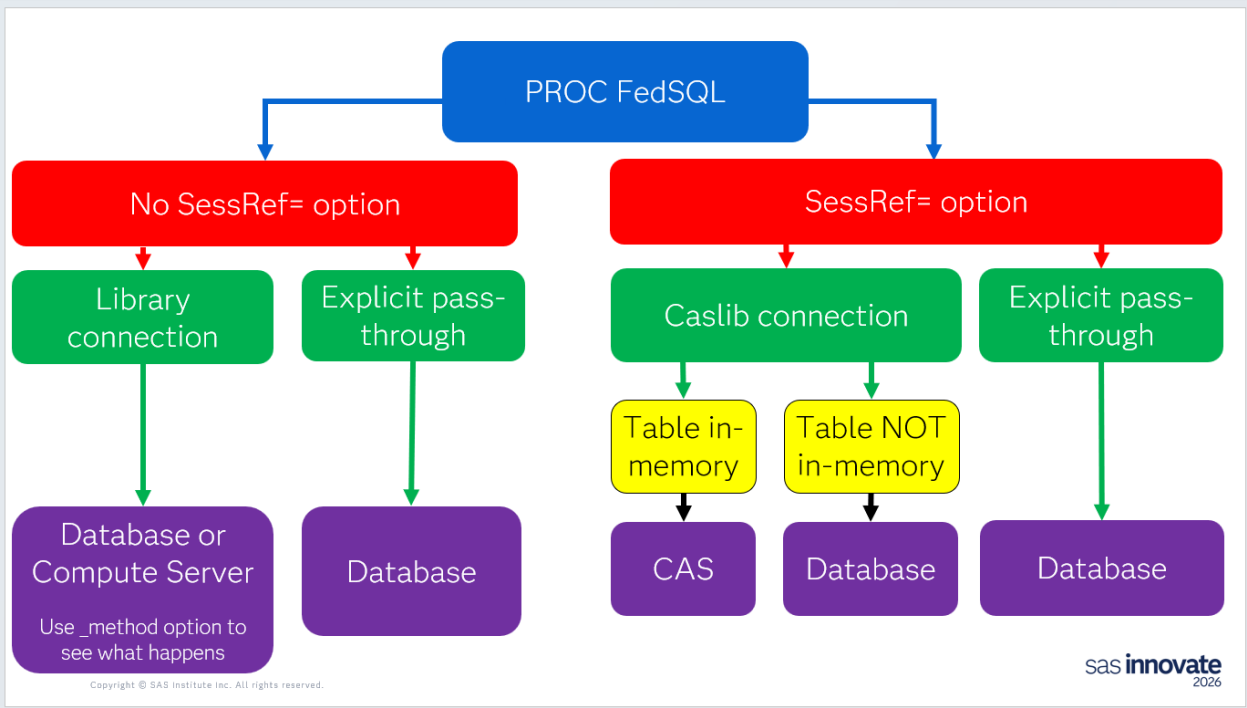
Variable	Type	Brief description
Category	Char	Type of loan, ex: “Consolidation”, “Credit Card”
Amount	Num	Size of loan in USD
Year	Num	Year account was opened
CancelledReason	Char	Why loan was cancelled
LastPurchase	Num	A DateTime for the last time the credit card was used. If Category ^= “Credit Card”, LastPurchase is missing
LoanGrade	Char	Previous model’s evaluation of loan, A (best) through E (worst)

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

Slides

When querying a database table in SAS, the location that processing occurs depends on several factors. The demo will walk us through examples of how those factors change processing. The following flow charts summarize how the location of processing is determined.



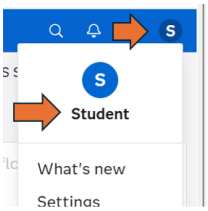


Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

Hands On Workshop Instructions

Setup: Sign In and Create Data

1. Open Google Chrome and select the **SASStudio** bookmark
2. If necessary, sign in to SAS Viya using the username *student* and the password *Metadata0*
   - **Note**: If you were already signed in to SAS Viya, make sure that you are signed in to the Student account. You can check the account by going to the top right of SAS Studio and clicking the circle icon, then view the account name. If the account isn't Student, log out and log back in using the required information.



3. When prompted to opt in to all of your assumable groups, select **No**
4. In SAS Studio, select the SAS Server pane (left side of the screen). Navigate to **Files** > **Home** > **workshop** > **SIWSQV**
5. From the SIWSQV folder, open **cre8data – Analyzing Database Tables.sas**. Run the entire program. The specifics of this program are outside the scope of this workshop. We only need to confirm it ran as intended. View the log, and confirm no error messages. View the results, and confirm the table named **LOANS_RAW** appears in the **Results from table.fileInfo**.

Note: There are preconditions for cre8data, including an Oracle database having been set up and an authDomain having been established for it. During SAS Innovate 2026, these will have been done in the lab for you

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

PROC SQL and SAS Compute Server Database Processing

1. In the **SIWSQV** folder, open the **01 – PROC SQL and Compute Server.sas** program. The LIBNAME statement connects the Oracle database to the SAS compute server using SAS/ACCESS Interface to Oracle. Run the statement.

```
libname or_db oracle path="//server.demo.sas.com:1521/ORCL"
                     authDomain="OracleAuth"
                     schema="STUDENT";
```

2. The OPTIONS statement turns on options to enable us to see what SQL was sent to the Oracle database for processing. Run the statement.

```
options sasTrace=',,,d' sasTraceLoc=sasLog noStSuffix;
```

3. In the IMPLICIT PASS-THROUGH ON THE SAS COMPUTE SERVER section, we will use SAS implicit pass-through to process the Oracle database table. With implicit pass-through, SAS attempts to convert SAS PROC SQL syntax into database SQL wherever possible. If it can't convert the SQL to database SQL, it brings the data to the SAS compute server for processing.

   a. Run the CONTENTS procedure to view all available tables in the Oracle database.

```
proc contents data=or_db._all_ noDs;
run;
```

View the results. Notice that there is a **LOANS_RAW** table. It is ok if other tables are listed, but they will not be used in this workshop.

View the log. After the text of the PROC CONTENTS, notice the lines beginning *ORACLE_X: Prepared*. *ORACLE_X* tells us the step's library is connected to Oracle, and *X* will vary depending on how many queries have been submitted in this session. *Prepared* tells us that the next row begins a proposed query to send to Oracle. After the prepared query, there is another pair of lines. *Oracle_X: Executed* tells us that a query executed on Oracle. The next line tells us which query. In the example log, the *ORACLE_1* query executed on Oracle.

The conversion and execution of non-Oracle code into an Oracle SQL query is called implicit pass-through.

```
ORACLE_1: Prepared: on connection 1
SELECT OBJECT_NAME ,OBJECT_TYPE  FROM  ALL_OBJECTS  OBJ
   WHERE (OBJ.OWNER ='STUDENT') AND (OBJ.OBJECT_TYPE  IN
   ('TABLE', 'VIEW'))
```

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

```
ORACLE_2: Executed: on connection 1
SELECT statement  ORACLE_1
```

b. The first PROC SQL query previews 10 rows from the **LOANS_RAW** database table. Run the SAS SQL procedure to run the query with the OBS= SAS data set option.

```
proc sql;
select *
   from or_db.loans_raw(obs=10);
quit;
```

View the log. Notice that implicit pass-through converted the SAS SQL query to an Oracle SQL query.

```
ORACLE_3: Prepared: on connection 0
SELECT * FROM STUDENT.LOANS_RAW FETCH FIRST   10 ROWS ONLY

ORACLE_4: Executed: on connection 0
SELECT statement  ORACLE_3
```

c. Next, we will count the total number of rows and total loan amount by **Category** in the **LOANS_RAW** database table. The first query will use implicit pass-through. The second query will disable implicit pass-through using the NOIPASSTHRU option. Both queries will produce identical results. Run both queries.

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

```
proc sql;
select Category,
       count(*) as TotalLoansByCategory format=comma16.,
       sum(Amount) as TotalAmount format=dollar20.2
    from or_db.loans_raw
    group by Category
    order by Category;
quit;

proc sql noIPassThru;
 select Category,
       count(*) as TotalLoansByCategory format=comma16.,
       sum(Amount) as TotalAmount format=dollar20.2
     from or_db.loans_raw
     group by Category
     order by Category;
quit;
```

View the log. Notice that the first query was converted into Oracle SQL via implicit pass-through. In the sample log, it ran in approximately three seconds.

```
ORACLE_5: Prepared: on connection 0
SELECT * FROM STUDENT.LOANS_RAW

ORACLE_6: Prepared: on connection 0
 select TXT_1."Category", COUNT(*) as
TotalLoansByCategory, SUM(TXT_1."Amount") as TotalAmount
from STUDENT.LOANS_RAW TXT_1 group by TXT_1."Category"
order by TXT_1."Category" asc NULLS FIRST

ORACLE_7: Executed: on connection 0
SELECT statement  ORACLE_6
...
NOTE: PROCEDURE SQL used (Total process time):
      real time           3.34 seconds
```

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

Category	TotalLoansByCategory	TotalAmount
Car Loan	785,883	$22,968,273,990.01
Consolidation	956,983	$19,611,577,759.59
Credit Card	5,944,360	$29,838,760,839.09
Education	448,928	$16,710,940,537.78
Home Improvement	672,629	$4,156,437,799.09
Major Purchase	112,124	$669,326,403.92
Medical	336,312	$12,097,338,287.34
Mortgage	855,370	$318,341,884,551.08
Moving Expenses	55,744	$333,383,431.31
Personal	111,949	$668,522,275.42
Small Business	783,945	$15,849,337,182.12
Vacation	112,218	$672,689,613.81
Weddings	55,978	$334,518,072.38

The log for the second query shows that even though implicit pass-through was disabled, SAS efficiently brings back only the necessary columns to process the query on the SAS compute server. In the sample log, the second query took approximately 6.5 seconds to process, longer than the implicit pass-through query for the same results.

```
ORACLE_9: Prepared: on connection 0
SELECT  "Category", "Amount" FROM STUDENT.LOANS_RAW

ORACLE_10: Executed: on connection 0
SELECT statement  ORACLE_9

151  quit;
NOTE: The PROCEDURE SQL printed page 8.
NOTE: PROCEDURE SQL used (Total process time):
      real time           6.54 seconds
```

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

Category	TotalLoansByCategory	TotalAmount
Car Loan	785,883	$22,968,273,990.01
Consolidation	956,983	$19,611,577,759.59
Credit Card	5,944,360	$29,838,760,839.09
Education	448,928	$16,710,940,537.78
Home Improvement	672,629	$4,156,437,799.09
Major Purchase	112,124	$669,326,403.92
Medical	336,312	$12,097,338,287.34
Mortgage	855,370	$318,341,884,551.08
Moving Expenses	55,744	$333,383,431.31
Personal	111,949	$668,522,275.42
Small Business	783,945	$15,849,337,182.12
Vacation	112,218	$672,689,613.81
Weddings	55,978	$334,518,072.38

d. In the next two queries, we will count the number of canceled loans by **Year,** and we'll filter for only loans that have a **CancelledReason** that begins with the word *Bad*. The first query uses the SAS SCAN function to search for the word *Bad*. The second query uses the ANSI standard LIKE operator. Both queries will achieve the same results. Run both queries.

```
proc sql;
 select Year,
        count(*) as TotalCancelled_BAD format=comma16.
    from or_db.loans_raw
    where scan(CancelledReason,1) = 'Bad'
    group by Year
    order by Year desc;
quit;

proc sql;
 select Year,
         count(*) as TotalCancelled_BAD format=comma16.
    from or_db.loans_raw
    where CancelledReason like 'Bad %'
    group by Year
    order by Year desc;
quit;
```

View the log and results. Notice that the first query attempts to use implicit pass-through, but it could not convert the query to Oracle SQL. Instead, it wrote a query to bring only the necessary columns to the SAS compute server for processing. In the sample log, this query is ORACLE_12. In the sample log, it took approximately four seconds to execute.

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

```
SAS_SQL:  Unable to convert the query to a DBMS specific
SQL statement due to an error. ACCESS ENGINE:  SQL
statement was not passed to the DBMS, SAS will do the
processing.

ORACLE_12: Prepared: on connection 0
SELECT  "Year", "CancelledReason" FROM STUDENT.LOANS_RAW

ORACLE_13: Executed: on connection 0
SELECT statement  ORACLE_12

170  quit;
NOTE: The PROCEDURE SQL printed page 9.
NOTE: PROCEDURE SQL used (Total process time):
      real time           4.37 seconds
```

Year	TotalCancelled_BAD
2025	2,979
2024	2,358
2023	1,889
2022	2,476
2021	2,241
2020	1,738
2019	1,221
2018	981
2017	668
2016	335

View the log for the second query. Notice that the SQL procedure syntax was converted into Oracle SQL through SAS implicit pass-through and the SELECT statement was passed to the Oracle database for processing. Only the smaller, summarized results were returned to SAS. In the sample log, this query ran in approximately two seconds, less than half the time that it took the previous query to run.

```
ORACLE_15: Prepared: on connection 0
 select TXT_1."Year", COUNT(*) as TotalCancelled_BAD from
  STUDENT.LOANS_RAW TXT_1 where TXT_1."CancelledReason"
  like 'Bad %' group by TXT_1."Year" order by
  TXT_1."Year" desc NULLS LAST
```

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

```
ORACLE_16: Executed: on connection 0
SELECT statement  ORACLE_15

ACCESS ENGINE:  SQL statement was passed to the DBMS for
  fetching data.
183  quit;
NOTE: The PROCEDURE SQL printed page 10.
NOTE: PROCEDURE SQL used (Total process time):
      real time           2.00 seconds
```

Year	TotalCancelled_BAD
2025	2,979
2024	2,358
2023	1,889
2022	2,476
2021	2,241
2020	1,738
2019	1,221
2018	981
2017	668
2016	335

e. The last query in this section uses the SAS YEAR and DATEPART functions to summarize by year of the **LastPurchase** datetime column. Run the query.

```
proc sql;
select year(datepart(LastPurchase)) as
LastPurchaseYear,
       count(*) as Total format=comma16.,
       count(*)/(select count(*)
                     from or_db.loans_raw
                     where Category = 'Credit Card')
          as LastPurchasePct format=percent7.1
    from or_db.loans_raw
    where Category = 'Credit Card'
    group by LastPurchaseYear
    order by Total desc;
quit;
```

View the log. Notice that SAS implicit pass-through was unable to convert the entire query to database SQL. Though it states this is “due to an error”, it is not a showstopping issue. It means the query had a feature it was unable to translate, in this case the DATEPART and YEAR functions.

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

While the entire query is not converted to Oracle SQL, implicit pass-through converted the subquery (ORACLE_19) to Oracle SQL to run in the database. It also converted the main query (ORACLE_20) to select only the necessary columns (**LastPurchase** and **Category**) and rows (Category = "*Credit Card"*) to bring back to the SAS compute server for processing. In the sample log, this query took approximately 8 seconds to run.

```
SAS_SQL:  Unable to convert the query to a DBMS specific
   SQL statement due to an error.
ACCESS ENGINE:  SQL statement was not passed to the DBMS,
   SAS will do the processing.

ORACLE_19: Prepared: on connection 0
 select COUNT(*) from STUDENT.LOANS_RAW TXT_2 where
   TXT_2."Category" = 'Credit Card'

ORACLE_20: Prepared: on connection 0
SELECT  "LastPurchase", "Category" FROM STUDENT.LOANS_RAW
   WHERE  ("Category" = 'Credit Card' )

ORACLE_21: Executed: on connection 0
SELECT statement  ORACLE_20

ORACLE_22: Executed: on connection 0
SELECT statement  ORACLE_19

ACCESS ENGINE:  SQL statement was passed to the DBMS for
   fetching data.
209  quit;
NOTE: The PROCEDURE SQL printed page 11.
NOTE: PROCEDURE SQL used (Total process time):
      real time           7.58 seconds
```

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

LastPurchaseYear	Total	LastPurchasePct
2025	4,937,417	83.1%
2024	365,633	6.2%
2023	243,853	4.1%
2022	167,060	2.8%
2021	103,472	1.7%
2020	61,937	1.0%
2019	35,952	0.6%
2018	19,221	0.3%
2017	7,880	0.1%
2016	1,935	0.0%

4. In the EXPLICIT PASS-THROUGH ON THE SAS COMPUTE SERVER section, we will use explicit pass-through to write and submit Oracle SQL using the SQL procedure. This enables us to use database features, and it will ensure that the query runs inside the database.
    - The following query uses Oracle SQL (explicit pass-through) to achieve the same results as the previous query. Run the query.

```
proc sql;
/* Connect to the Oracle database */
connect using or_db;

/* Use SAS formats for the results from the Oracle query */
select LastPurchaseYear,
        Total format=comma16.,
       LastPurchasePct format=percent7.1
   from connection to or_db
    (
     select EXTRACT(YEAR FROM "LastPurchase") as
                LastPurchaseYear,
           count(*) as Total,
           count(*)/(select count(*)
                       from loans_raw
                       where "Category" = 'Credit Card')
                     as  LastPurchasePct
        from loans_raw
        where "Category" = 'Credit Card'
        group by EXTRACT(YEAR FROM "LastPurchase")
        order by Total desc
    );

/* Disconnect from the Oracle database */
disconnect from or_db;
quit;
```

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

View the log. Notice that SAS sent the Oracle SQL query directly to the database for processing. In the sample log, this explicit pass-through query ran in approximately four seconds, less than the equivalent implicit pass-through query in part 3e.

```
ORACLE_23: Prepared: on connection 2
select EXTRACT( YEAR FROM "LastPurchase") as
  LastPurchaseYear, count(*) as Total, count(*)/(select
  count(*) from loans_raw where "Category" = 'Credit
  Card') as LastPurchasePct from loans_raw where
  "Category" = 'Credit Card' group by EXTRACT(YEAR FROM
  "LastPurchase") order by Total desc


ORACLE_24: Executed: on connection 2
SELECT statement  ORACLE_23
...
NOTE: The PROCEDURE SQL printed page 12.
NOTE: PROCEDURE SQL used (Total process time):
      real time           4.18 seconds
```

LASTPURCHASEYEAR	TOTAL	LASTPURCHASEPCT
2025	4,937,417	83.1%
2024	365,633	6.2%
2023	243,853	4.1%
2022	167,060	2.8%
2021	103,472	1.7%
2020	61,937	1.0%
2019	35,952	0.6%
2018	19,221	0.3%
2017	7,880	0.1%
2016	1,935	0.0%

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

PROC FedSQL and SAS Compute Server Database Processing

1. In the **SIWSQV folder**, open the **02 - FedSQL and Compute Server.sas** program. The LIBNAME statement connects the Oracle database to the SAS compute server using SAS/ACCESS Interface to Oracle. Run the statement.

```
libname or_db oracle path="//server.demo.sas.com:1521/ORCL"
                     authDomain="OracleAuth"
                     schema="STUDENT";
```

2. PROC FedSQL can be used with the compute server. When the data source is a SAS library, implicit pass through can be used.

   a. By default, no pass through information is placed in the log for PROC FedSQL. The OPTIONS statement that places text in the log for PROC SQL does not work with PROC FedSQL. Run the OPTIONS statement and FedSQL query.

```
options sasTrace=',,,d' sasTraceLoc=sasLog noStSuffix;

proc fedsql;
select distinct LoanGrade
   from or_db.loans_raw
   order by LoanGrade
;
quit;
```

   View the log. Notice that it only says PROCEDURE FEDSQL was used.

```
NOTE: The PROCEDURE FEDSQL printed page 59.
NOTE: PROCEDURE FEDSQL used (Total process time):
      real time           2.38 seconds
      cpu time            0.07 seconds
```

LoanGrade
A
B
C
D
E

   b. The _method option on the PROC FedSQL statement places text in the log for that step. Run the query.

```
proc fedsql _method;
```

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

```
select distinct LoanGrade
   from or_db.loans_raw
   order by LoanGrade
;
quit;
```

View the log. The numbering on the left column may vary. Notice the text “Methods: Full query pushdown!”. That means the query ran entirely in the database, and only the results were passed back to the compute server.

```
80   proc fedsql _method;
81   select distinct LoanGrade
82      from or_db.loans_raw
83      order by LoanGrade
84   ;

Methods: Full query pushdown!
85   quit;
NOTE: The PROCEDURE FEDSQL printed page 60.
NOTE: PROCEDURE FEDSQL used (Total process time):
      real time           2.36 seconds
      cpu time            0.08 seconds
```

LoanGrade
A
B
C
D
E

c. Not every query in PROC FedSQL runs entirely in the database. Run the query.

```
proc fedsql _method;
select distinct LoanGrade
   from or_db.loans_raw
   where scan(cancelledReason, 1) = 'Bad'
   order by LoanGrade
;
quit;
```

View the log. The numbering on the left column may vary.

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

Because the text does not contain “Full query pushdown!”, we know that at least part of the query ran in SAS.

```
81   proc fedsql _method;
82   select distinct LoanGrade
83      from or_db.loans_raw
84      where scan(cancelledReason, 1) = 'Bad'
85      order by LoanGrade
86   ;

Methods:
Number of Sorts Performed is : 1
        Sort
          SeqScan with qual from OR_DB.STUDENT.LOANS_RAW
87   quit;
NOTE: The PROCEDURE FEDSQL printed page 61.
NOTE: PROCEDURE FEDSQL used (Total process time):
      real time           6.13 seconds
      cpu time            2.85 seconds
```

LoanGrade
A
B
C
D
E

3. In the EXPLICIT PASS-THROUGH WITH FEDSQL ON THE SAS COMPUTE SERVER section, we will use explicit pass-through to write and submit Oracle SQL using the FedSQL procedure. This enables us to use database features, and it will ensure that the query runs inside the database. _method is not needed, because everything in the parentheses of the FROM clauses either runs in the database, or errors out. Run the query.

```
proc fedsql;
create table work.example as
select *
    from connection to or_db
    (select EXTRACT( YEAR FROM "LastPurchase")
                    as LastPurchaseYear,
            count(*) as Total,
            count(*)/(select count(*)
```

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

```
                            from loans_raw
                            where "Category" = 'Credit Card')
                    as LastPurchasePct
            from loans_raw
            where "Category" = 'Credit Card'
            group by EXTRACT(YEAR FROM "LastPurchase")
            order by Total desc
        )
;
quit;
```

View the log. The numbering on the left column may vary.

Because the query uses explicit pass-through and did not error out, we know that the portion inside of the parentheses in the FROM clause ran in the database. The result was passed to SAS, and SAS ran the remainder of the query.

```
95       )
96   ;
NOTE: Execution succeeded. 10 rows affected.
97   quit;
NOTE: PROCEDURE FEDSQL used (Total process time):
      real time           4.33 seconds
      cpu time            0.05 seconds
```

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

CAS Server Database Processing

1. Open the **03 - CAS Server.sas** program. The CAS statement makes a connection to the CAS server from the compute server. The CASLIB statement creates a CAS server connection to the same Oracle database that we used earlier. Run the CAS and CASLIB statements.

```
cas conn;

caslib ordb_cas dataSource=(srctype="oracle",
                            authDomain="OracleAuth",
                            path="//server.demo.sas.com:1521/ORCL",
                            schema="STUDENT");
```

2. View available data source files and in-memory CAS tables in the **ordb_cas** caslib by running the CASUTIL procedure with the LIST FILES and LIST TABLES statements. Run the step.

```
proc casUtil inCaslib = 'ordb_cas';
   list files;   * View available database tables *;
   list tables;  * View available in-memory CAS tables *;
quit;
```

View the results and log. Notice that we have access to the same Oracle database with the **LOANS_RAW** table. We also see that no tables are loaded into memory on the CAS server.

The CASUTIL Procedure

Caslib Information	
Library	ORDB_CAS
Source Type	oracle
AuthenticationDomain	OracleAuth
Session local	Yes
Active	Yes
Personal	No
Hidden	No
Transient	No
TableRedistUpPolicy	Not Specified
Schema	STUDENT
Path	//server.demo.sas.com:1521/ORCL

The CASUTIL Procedure

CAS File Information

Name	Catalog	Schema	Type	Description
CUSTOMERS	ORDB_CAS	STUDENT	TABLE	
LOANS_RAW	ORDB_CAS	STUDENT	TABLE	

```
NOTE: No tables are available in caslib ORDB_CAS of Cloud Analytic Services.
```

3. Next, we will use implicit pass-through in CAS using a caslib. This process is similar to the compute server process. The main difference is that we will use the FEDSQL procedure with the

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

SESSREF= option instead of the SQL procedure. For implicit pass-through to be used, we must reference the database table name, not an in-memory CAS table. In this example, **ordb_cas.loans_raw** is not in-memory in CAS. It is in Oracle. Run the FEDSQL procedure.

```
proc fedSql sessref=conn;
select Category,
       count(*)as TotalLoansByCategory,
        sum(Amount) as TotalAmount
     from ordb_cas.loans_raw
     group by Category;
quit;
```

View the log and results. The log shows that the SQL statement was fully offloaded to the underlying data source via full pass-through. Because there is no ORDER BY clause in the query, the rows of your results may be in a different order than the sample.

```
NOTE: The SQL statement was fully offloaded to the underlying data source via full pass-
through
```

Category	TOTALLOANSBYCATEGORY	TOTALAMOUNT
Major Purchase	112124	669326404
Weddings	55978	334518072
Moving Expenses	55744	333383431
Mortgage	855370	318341884551
Vacation	112218	672689614
Personal	111949	668522275
Credit Card	5944360	29838760839
Home Improvement	672629	4156437799
Consolidation	956983	19611577760
Small Business	783945	15849337182
Medical	336312	12097338287
Car Loan	785883	22968273990
Education	448928	16710940538

4. In this next example, we will use explicit pass-through with a caslib to subset the database table using Oracle SQL. The syntax for explicit pass-through in FEDSQL is similar to that used in PROC SQL, except CONNECT and DISCONNECT statements are not needed in FEDSQL. FEDSQL uses the connection information that is already stored in the caslib. In this example, the SAS CREATE TABLE statement creates an in-memory CAS table with the results from Oracle. Run the FEDSQL procedure.

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

```
proc fedSql sessRef=conn;
create table ordb_cas.CCAccounts2022{options replace=true} as
  select *
  from connection to ordb_cas
     (
   /* Send native Oracle query directly to the database for
      processing */
      select *
        from loans_raw
        where EXTRACT(YEAR FROM "LastPurchase") = 2022 and
              "Category" = 'Credit Card'
     );
quit;
```

View the log. It does not mention offloading. We know the query was run in Oracle, because explicit pass-through either runs in the database or errors out. This did not error out. It made (or overwrote) CCACCOUNTS2022 as an in-memory table in the ORDB_CAS caslib.

```
NOTE: Table CCACCOUNTS2022 was created in caslib ORDB_CAS with 167060 rows returned.
```

5. In the Analyze a CAS table section, we will process an in-memory CAS table using a variety of methods. This builds off of part 4. If part 4 has not run and created **ordb_cas.CCAccounts2022**, run it now.

   a. First, the CASUTIL procedure uses the LIST TABLES statement to list available CAS tables in the **ordb_cas** caslib. The CONTENTS statement shows the metadata of the **CCAccounts2022** CAS table that we created earlier. Run the CASUTIL procedure.

```
proc casutil;
    list tables incaslib='ordb_cas';
    contents casdata="CCAccounts2022" incaslib="ordb_cas";
quit;
```

   b. Once a file is loaded into CAS, it can be processed in-memory in CAS. We can use the FEDSQL procedure with the SESSREF= option to execute SQL queries on a CAS table. Here, we run two queries on the CAS table, and the processing is done in CAS. Run the FEDSQL procedure.

```
proc fedsql sessref=conn;
 select *
    from ordb_cas.CCAccounts2022
    limit 10;

select LoanGrade,
        count(*) as TotalLoansByCategory,
        sum(Amount) as TotalAmount
```

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

```
     from ordb_cas.CCAccounts2022
     group by LoanGrade
     order by LoanGrade;
quit;
```

View the log. Since an in-memory CAS table and sessRef= were used, the queries either ran in CAS or errored out. These did not error out, so we know they ran in CAS.

c. Once a table is loaded into memory, we can also execute CAS actions to process the data. For example, we can use the simple.summary CAS action to obtain descriptive statistics. Run the CAS procedure.

```
proc cas;
simple.summary /
     table = {name = 'CCAccounts2022',
              caslib = 'ordb_cas'};
quit;
```

View the log and results. In the log, notice that the summary action is processed in under half a second and returns a variety of descriptive statistics. In the results, note the variety of summary statistics generated.

Descriptive Statistics for CCACCOUNTS2022

Column	Minimum	Maximum	N	Sum	Mean	Std Dev	Std Error	Variance	Coeff of Variation	Corrected SS	USS	t Value	Pr > |t|	N Miss	Skewness	Kurtosis
Year	2016.00	2022.00	167060	337500231	2020.23	1.5531	0.003800	2.4121	0.07688	402964	6.818E11	531667	<.0001	0	-0.8452	-0.02721
Month	1.0000	12.0000	167060	1047573	6.2706	3.1846	0.007792	10.1418	50.7862	1694281	8263233	804.80	<.0001	0	0.07789	-1.1488
Day	1.0000	27.0000	167060	2335225	13.9784	7.5127	0.01838	56.4399	53.7449	9428799	42071417	760.50	<.0001	0	0.001189	-1.1911
Amount	-1253.79	31098	167060	194994427	1167.21	2665.78	6.5221	7106389	228.39	1.187E12	1.415E12	178.96	<.0001	0	2.4063	5.7202
InterestRate	8.4900	32.4700	167060	2958013.8	17.7063	2.8435	0.006957	8.0854	16.0592	1350736	53726204	2545.15	<.0001	0	0.4514	0.1245
LoanLength	999999	999999	167060	1.6706E11	999999	0	0	0	0	0	1.671E17	.	.	0	.	.
LastPurchase	1.9566E9	1.9882E9	167060	3.2972E14	1.9737E9	9026291	22084	8.147E13	0.4573	1.361E19	6.508E23	89371.7	<.0001	0	-0.1720	-1.1506
Cancelled	0	1.0000	167060	1671	0.01000	0.09951	0.000243	0.009902	994.87	1654.29	1671.00	41.08	<.0001	0	9.8482	94.9890
Promotion	0	0	167060	0	0	0	0	0	.	0	0	.	.	0	.	.

6. Sometimes, people try to use PROC SQL to run queries on an in-memory CAS table. While this is not recommended, it is possible for smaller tables.

   To execute PROC SQL on a CAS table, we first need to create a library reference to a caslib with the CAS engine using the LIBNAME statement. Then, the library can be used to point to in-memory tables of the caslib. During the query, the CAS table is temporarily copied to the compute server for processing. This is done in the background. If the copy is successful, no notes are placed in the log. Run the LIBNAME statement and the SQL procedure.

```
libname ordb_cas cas caslib='ordb_cas';

proc sql;
select Category,
       count(*) as TotalLoansByCategory,
       sum(Amount) as TotalAmount
```

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

```
    from ordb_cas.CCAccounts2022
    group by Category;
quit;
```

Category	TotalLoansByCategory	TotalAmount
Credit Card	167060	1.9499E8

This CAS table was small enough that it was temporarily copied to the compute server without issue.

7. To show what happens if we attempt PROC SQL with a large CAS table, we must first create a sufficiently large in-memory table in CAS. Then, we will then attempt to query it using PROC SQL. Run the CASUTIL and SQL procedures.

```
proc casUtil;
  load inCaslib='ordb_cas' casData="LOANS_RAW"
      outCaslib='ordb_cas' casOut="loans_raw_in_memory" replace;
quit;

proc sql;
select Category,
       count(*) as TotalLoansByCategory,
       sum(Amount) as TotalAmount
    from ordb_cas.loans_raw_in_memory
    group by Category;
quit;
```

View the log. Notice that we received an error, because **loans_raw_in_memory** is larger than the DATALIMIT option. The table was too large to background copy to compute.

```
ERROR: The maximum allowed bytes (104857600) of data have been fetched from Cloud Analytic
Services. Use the DATALIMIT option to increase the maximum value.
```

8. If you see the previous error, move the query to PROC FEDSQL with SESSREF=. This has been done for us in the next PROC FEDSQL step. Notice this query is identical to the previous PROC SQL query. If the original query had non-ANSI standard features (like the calculated keyword), then additional adjustments would need to be made. Run the FEDSQL procedure.

```
proc fedSql sessRef=conn;
select Category,
       count(*) as TotalLoansByCategory,
       sum(Amount) as TotalAmount
```

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

```
    from ordb_cas.loansA_raw_in_memory
    group by Category;
quit;
```

View the log and results. There should be no errors or warnings. Because the query ran with sessRef=conn and on an in-memory CAS table, it ran in CAS.

Category	TOTALLOANSBYCATEGORY	TOTALAMOUNT
Personal	111949	668522275
Credit Card	5944360	29838882797
Medical	336312	12097338287
Small Business	783945	15849337182
Vacation	112218	672689614
Education	448928	16710940538
Mortgage	855370	318341884551
Moving Expenses	55744	333383431
Weddings	55978	334518072
Home Improvement	672629	4156437799
Car Loan	785883	22968273990
Consolidation	956983	19611577760
Major Purchase	112124	669326404

Copyright © 2026, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.